

Архитектура кода на Python

YOUNG & YANDEX

Дамир Гарифуллин
Бэкенд-разработчик карт офисов



Содержание

01 Принципы написания хорошего кода

02 Паттерны проектирования

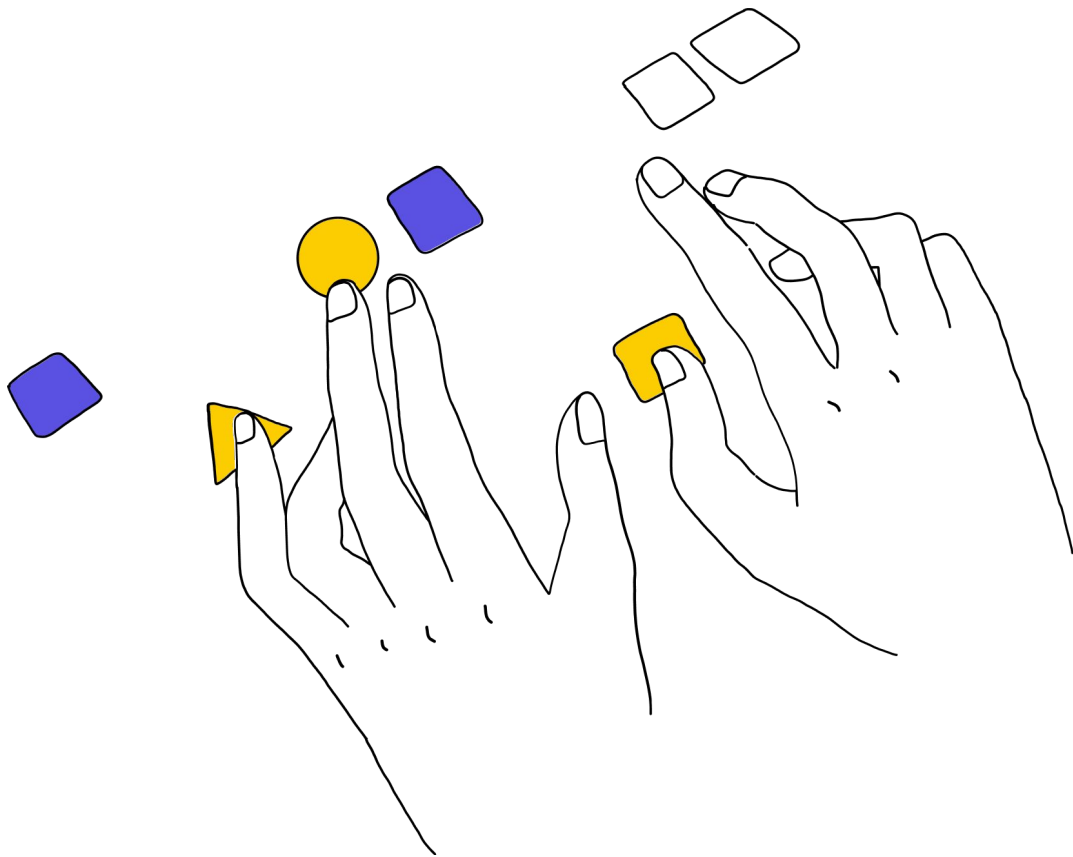
03 Трехслойная архитектура



Принципы написания кода



Python - объектно-ориентированный язык



Чистый код



- Легко читается
- Понятен любому разработчику
- Прост в поддержке

Почему “плохой” код плохой?

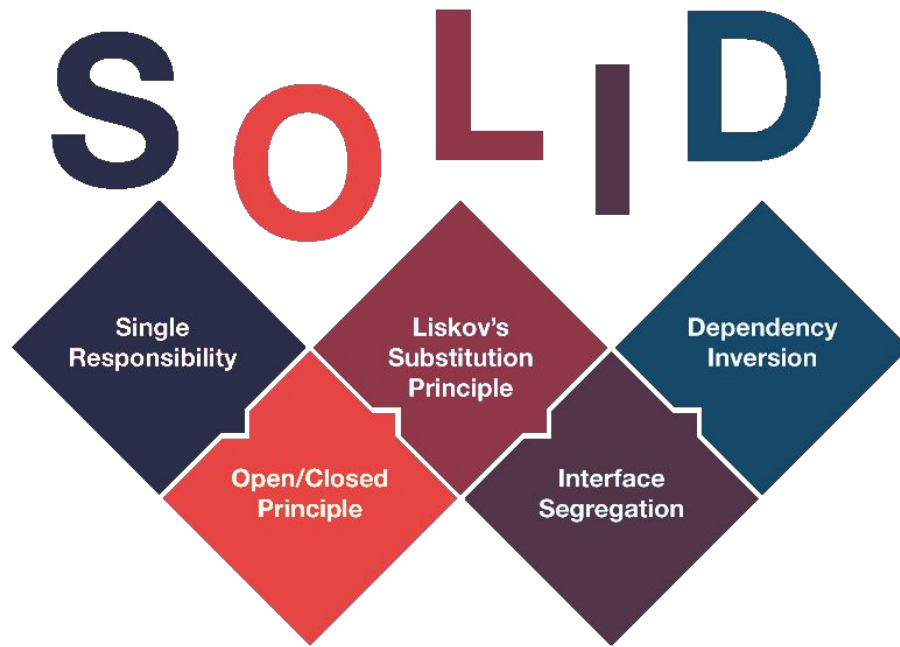
- Чтобы внести изменения в некачественный код, нужно много времени и это стоит дорого
- Добавить что-то новое сложно
- Код непредсказуем
- Люди не любят работать с таким кодом

SOLID

Роберт Мартин (дядя Боб) - автор основных принципов проектирования объектно-ориентированного проектирования и программирования.

Автор книг «Чистый код. Создание, анализ и рефакторинг», «Чистая архитектура. Искусство разработки программного обеспечения»





Разберем на примере

Задача - уведомить пользователя по электронной почте о том, что в вашей системе произошли изменения

Single Responsibility

ТАК НЕЛЬЗЯ



Принцип единой ответственности говорит о том, что у каждого объекта есть своя ответственность и эта ответственность у него только одна.

использов
письма

```
class UserNotifier:
    def notify_user_about_changes(user: User):
        to_email = user.mail
        from_email = 'from@gmail.com'
        subject = 'This is a email with a changes list!'
        message = MIMEMultipart()
        message['Subject'] = subject
        message['From'] = from_email
        message['To'] = to_email
        message.attach(MIMEText("Some text here", "html"))
        message_body = message.as_string()
```

Создание
почтового
клиента

```
→ { host = "127.0.0.1"
    port = 587
    mail_client = Client()
```

```
mail_client.send_message(host, port, message_body)
```

Отправка

Single Responsibility

Принцип единой ответственности говорит о том, что у каждого объекта есть своя ответственность и эта ответственность у него только одна.

```
class UserNotifier:
    def notify_user_about_changes(self, user: User):
        message_body =
            ① EmailGenerator.generate_mail_body(user.mail)
            ② mail_client = GmailClient.get_mail_client()
            ③ mail_client.send_message(message_body)

class EmailGenerator:
    @staticmethod
    def generate_mail_body(user_mail: str) -> str:
        to_email = user_mail
        from_email = 'from@gmail.com'
        subject = 'This is a email with a changes list!'
        message = MIMEMultipart()
        message['Subject'] = subject
        message['From'] = from_email
        message['To'] = to_email

        message.attach(MIMEText("Some text here", "html"))
        return message.as_string()

class GmailClient:
    @staticmethod
    def get_mail_client() -> Client:
        host = "127.0.0.1"
        port = 587
        return Client(host, port)
```



Open-closed Principle

Модули, отвечающие принципу открытости-закрытости, имеют два главных признака:

1. Открыты для расширения.
Это означает, что поведение модуля может быть расширено.
2. Закрыты для изменений.
Исходный код такого модуля неприкасаем.



Open-closed Principle

Модули, отвечающие принципу открытости-закрытости, имеют два главных признака:

1. Открыты для расширения.
Это означает, что поведение модуля может быть расширено.
2. Закрыты для изменений.
Исходный код такого модуля неприкасаем.

```
class UserNotifier:
    def notify_user_about_changes(
        self,
        user: User,
    ):
        message_body =
            EmailGenerator.generate_mail_body(user.mail)

        if isinstance(client, OutlookClient):
            client = OutlookClient.get_mail_client()
            client.send(message_body)
        elif isinstance(client, GmailClient):
            client = GmailClient.get_mail_client()
            client.send_message(message_body)
```

elif isinstance(client, YandexClient):

Очень плохо →

Open-closed Principle

Другой пример
абстрактного класса от GPT

```
class MailClient(ABC):  
    @abstractmethod  
    def send_mail(self, body: str):  
        pass
```

класс от которого
надо наследоваться
для абстр классов
оставление
пустой абстр-метод
а потом заполнен

```
class GmailClient(MailClient):  
    def __init__(self, client: Gmail):  
        self._client = client
```

```
    def send_mail(self, body: str):  
        self._client.send_message(body)
```

```
class OutlookClient(MailClient):  
    def __init__(self, client: Outlook):  
        self._client = client
```

```
    def send_mail(self, body: str):  
        self._client.send(body)
```

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):  
    @abstractmethod  
    def make_sound(self):  
        pass
```

```
    @abstractmethod  
    def move(self):  
        pass
```

```
class Dog(Animal):  
    def make_sound(self):  
        print("Woof!")  
  
    def move(self):  
        print("Dog is running")
```

```
class Cat(Animal):  
    def make_sound(self):  
        print("Meow!")  
  
    def move(self):  
        print("Cat is jumping")
```

protected →

Двойное подчеркивание (`\_\_`)

Двойное подчеркивание перед именем атрибута или метода приводит к его "именификации" (name mangling). Это означает, что атрибут становится частным (private) и не может быть легко доступен за пределами класса. Интерпретатор Python изменяет имя атрибута, добавляя к нему имя класса.

```
python
class MyClass:
    def __init__(self):
        self.__private_attr = 42 # Приватный атрибут

    def get_private_attr(self):
        return self.__private_attr

# Создание объекта класса
obj = MyClass()

# Попытка доступа к приватному атрибуту вызовет ошибку
# print(obj.__private_attr) # AttributeError

# Доступ к приватному атрибуту через именификацию
print(obj._MyClass__private_attr) # Правильный доступ
```

Одиночное подчеркивание (`\_`)

Когда имя атрибута начинается с одного подчеркивания (`\_`), это является конвенцией, которая указывает, что атрибут является защищенным (protected). Это означает, что он не должен использоваться напрямую за пределами класса и его подклассов. Однако это всего лишь конвенция, а не жесткое ограничение.

```
python
class MyClass:
    def __init__(self):
        self._protected_attr = 42 # Защищенный атрибут

    def get_protected_attr(self):
        return self._protected_attr

# Создание объекта класса
obj = MyClass()

# Доступ к защищенному атрибуту (не рекомендуется)
print(obj._protected_attr)
```

← private

Open-closed Principle



Способ : DA
это след. принцип

```
class MailClient(ABC):
    @abstractmethod
    def send_mail(self, body: str):
        pass

class GmailClient(MailClient):
    def __init__(self, client: Gmail):
        self._client = client

    def send_mail(self, body: str):
        self._client.send_message(body)

class OutlookClient(MailClient):
    def __init__(self, client: Outlook):
        self._client = client

    def send mail(self, body: str):
        self._client.send(body)
```

По себе можно
сделать MailClient

```
class UserNotifier:
    def __init__(
        self,
        email_generator: EmailGenerator,
        client: [GmailClient | OutlookClient],
    ):
        self.email_generator = email_generator
        self._client = client

    def notify_user_about_changes(
        self,
        user: User,
    ):
        message_body =
            self.email_generator.generate_mail_body(
                user.mail)
        self._client.send_mail(message_body)
```



Liskov Substitution Principle

Функции, которые используют базовый тип, должны иметь возможность использовать подтипы базового типа, не зная об этом.

```
class UserNotifier:
    def __init__(
        self,
        email_generator: EmailGenerator,
        client: [GmailClient | OutlookClient],
    ):
        self.email_generator = email_generator
        self._client = client

    def notify_user_about_changes(
        self,
        user: User,
    ):
        message_body =
            self._email_generator.generate_mail_body(
                user.mail)
        self._client.send_mail(message_body)
```

ПЛОХА!

Liskov Substitution Principle

Функции, которые используют базовый тип, должны иметь возможность использовать подтипы базового типа, не зная об этом.

При этом, любой дочерний класс должен вести себя точно так же, как родительский класс.

Родитель

```
class MailClient(ABC):  
    @abstractmethod  
    def send_mail(self, body: str):  
        pass
```

Дочерний

```
class GmailClient(MailClient):  
    def __init__(self, client: Gmail):  
        self._client = client  
  
    def send mail(self, body: str):  
        self._client.send_message(body)
```

Дочерний

```
class OutlookClient(MailClient):  
    def __init__(self, client: Outlook):  
        self._client = client  
  
    def send mail(self, body: str):  
        self._client.send(body)
```



Liskov Substitution Principle

```
class UserNotifier:
    def __init__(
        self,
        email_generator: EmailGenerator,
        client: MailClient,
    ):
        self.email_generator = email_generator
        self._client = client

    def notify_user_about_changes(
        self,
        user: User,
    ):
        message_body =
            self.email_generator.generate_mail_body(
                user.mail)
        self._client.send_mail(message_body)
```

Xopovo!

```
class MailClient(ABC):
    @abstractmethod
    def send_mail(self, body: str):
        pass
```

```
class GmailClient(MailClient):
    def __init__(self, client: Gmail):
        self._client = client

    def send_mail(self, body: str):
        self._client.send_message(body)
```

```
class OutlookClient(MailClient):
    def __init__(self, client: Outlook):
        self._client = client

    def send_mail(self, body: str):
        self._client.send(body)
```



Interface Segregation

Принцип разделения интерфейсов говорит о том, что слишком «толстые» интерфейсы необходимо разделять на более маленькие и специфические, чтобы программные сущности маленьких интерфейсов знали только о методах, которые необходимы им в работе.

Перевод
с eng на eng



```
class EmailGenerator(ABC):
    @abstractmethod
    @staticmethod
    def generate_mail_body(user_mail: str) -> str:
        pass

    @abstractmethod
    def translate_from_eng(self) -> str:
        pass

class EnglishEmailGenerator(EmailGenerator):
    @staticmethod
    def generate_mail_body(user_mail: str) -> str:
        ...

    def translate_from_eng(self) -> str:
        return str

class RussianEmailGenerator(EmailGenerator):
    @staticmethod
    def generate_mail_body(user_mail: str) -> str:
        ...

    def translate_from_eng(self) -> str:
        ...
```



Interface Segregation

Создание отдельных

generator →

translator

Принцип разделения интерфейсов говорит о том, что слишком «толстые» интерфейсы необходимо разделять на более маленькие и специфические, чтобы программные сущности маленьких интерфейсов знали только о методах, которые необходимы им в работе.

Не нужен translator

```
class EmailGenerator(ABC):  
    @abstractmethod  
    @staticmethod  
    def generate_mail_body(user_mail: str) -> str:  
        pass
```

```
class EmailTranslator(ABC):  
    @abstractmethod  
    def translate_from_eng(self) -> str:  
        pass
```

```
class EnglishEmailGenerator(EmailGenerator):  
    @staticmethod  
    def generate_mail_body(user_mail: str) -> str:  
        ...
```

```
class RussianEmailGenerator(  
    EmailGenerator, EmailTranslator  
):  
    @staticmethod  
    def generate_mail_body(user_mail: str) -> str:  
        ...  
  
    def translate_from_eng(lang: str) -> str:  
        ...
```

Здесь он уже есть



Dependency Inversion

1. Модули верхних уровней не должны зависеть от модулей нижних уровней. Оба типа модулей должны зависеть от абстракций.
2. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

Если захотим добавить
новый язык, то придется
добавлять еще два
класса

```
class EmailGenerator(ABC):  
    @abstractmethod  
    @staticmethod  
    def generate_mail_body(user_mail: str) -> str:  
        pass
```

```
class EmailTranslator(ABC):  
    @abstractmethod  
    def translate_from_eng(self) -> str:  
        pass
```

```
class EnglishEmailGenerator(EmailGenerator):  
    @staticmethod  
    def generate_mail_body(user_mail: str) -> str:  
        ...
```

```
class RussianEmailGenerator(  
    EmailGenerator, EmailTranslator  
):  
    @staticmethod  
    def generate_mail_body(user_mail: str) -> str:  
        ...  
  
    def translate_from_eng(lang: str) -> str:  
        ...
```

Dependency Inversion

Можно создать
генератор и переводчика

```
class EmailGeneratorWithTranslation(
    EmailGenerator,
):
    def __init__(
        self,
        translator: Optional[EmailTranslator] =
            None,
    ):
        self._translator = translator

    def generate_mail_body(
        self,
        user_mail: str,
    ) -> str:
        ...
        if self.translator:
            return self.translate(generator_result)

        return generator_result
```

```
class EmailGenerator(ABC):
    @abstractmethod
    @staticmethod
    def generate_mail_body(user_mail: str) -> str:
        pass
```

```
class EmailTranslator(ABC):
    def __init__(
        self,
        origin_lang: str,
        translation_lang: str
    ):
        self.origin_lang = origin_lang
        self._translation_lang = translation_lang

    @abstractmethod
    def translate(self, text: str) -> str:
        pass
```



KISS (Keep It Short and Simple)

Делай проще!



KISS: примеры

Именуй переменные и функции понятно.

Лучше назвать переменную `pretty_clear_variable`, чем сократить до непонятного `pcv`.

Соблюдай согласованность именования.

Если в одном месте назвал объект `“motorcycle”`, то не именуй такой же объект в соседнем файле как `“bike”`.

Не используй инструмент, если он тебе не нужен.

Ты точно уверен, что для ручки, в которую ходят 3 клиента в час, нужно прикрутить кэш?

KISS: примеры

Не сокращай названия во вред понятности.

Лучше назвать функцию `crop_and_resize_image()`, чем дать имя `process_image()` и словами описывать в docstrings что картинка будет обрезана и сжата или не описывать вообще.

Не изобретай велосипед.

Изучи стандартную библиотеку.
Погугли.

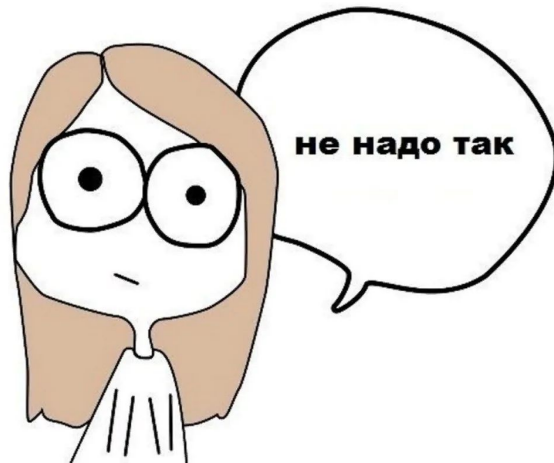
Если кто-то потратил время и силы на решение такой же проблемы, зачем тебе тратить драгоценное время?

KISS: примеры

Сделай код ленивым.

Если можешь в каких-то случаях не выполнять часть кода, то не выполняй.

```
def create_user(  
    name: str,  
    surname: str,  
    age: int,  
    db: Database,  
) -> None:  
    try:  
        db.save(name, surname, age)  
    except Exception:  
        raise ValueError()
```



KISS: примеры

Лучше сделай так:

```
def create_user(name: str, surname: str, age: int, db: Database) -> None:
    if not name or not surname:
        raise ValueError('empty name or surname')
    if age < 18:
        raise ValueError('age must be at least 18')
    db.save(name, surname, age)
```



YAGNI - You aren't gonna need it

1. Не пиши код про запас.
2. Не бойся удалять лишние неиспользуемые методы/переменные.

YAGNI - You aren't gonna need it

1. Не пиши код про запас.
2. Не бойся удалять лишние неиспользуемые методы/переменные.

Почему стоит это делать?

YAGNI - You aren't gonna need it

1. Не пиши код прозапас.
2. Не бойся удалять лишние неиспользуемые методы/переменные.

Почему стоит это делать?

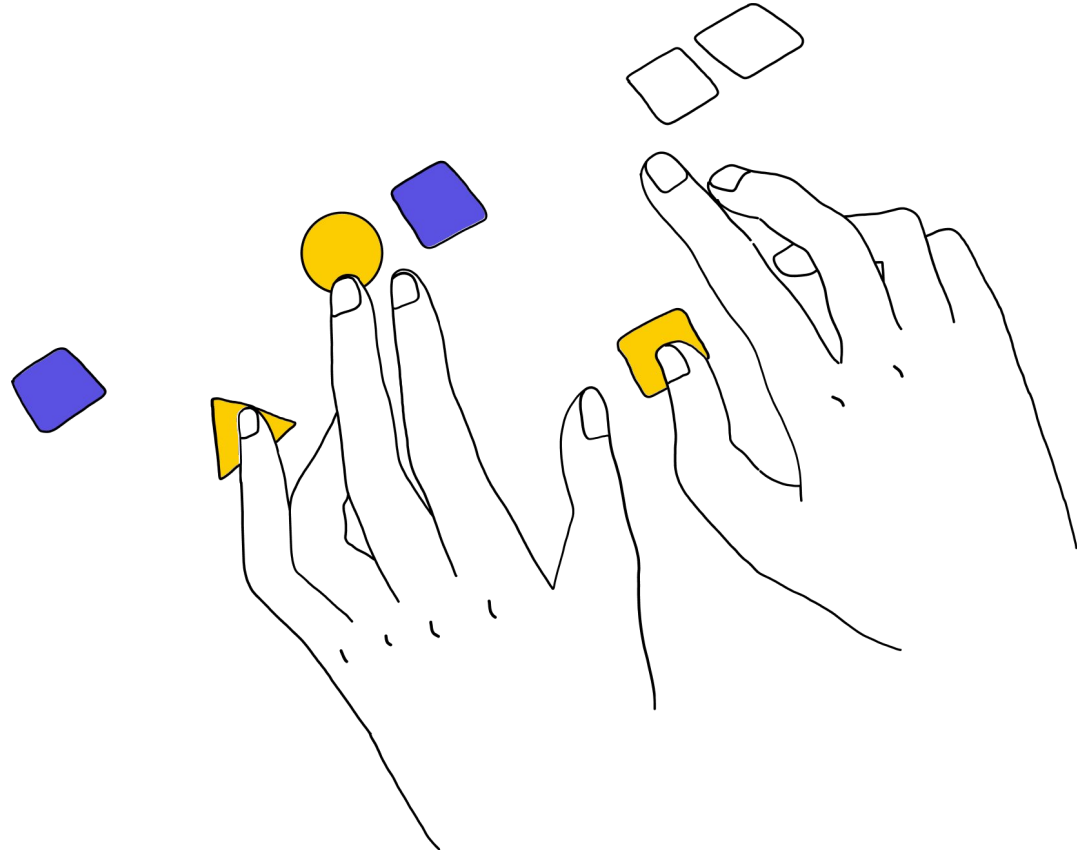
- Бóльшее количество кода - больше места для появления багов
- Требования меняются быстро, велик риск что код понадобится в другом виде или не понадобится вообще
- Меньше головоломок для разработчиков, которые случайно наткнутся на неиспользуемый код

DRY - Don't Repeat Yourself

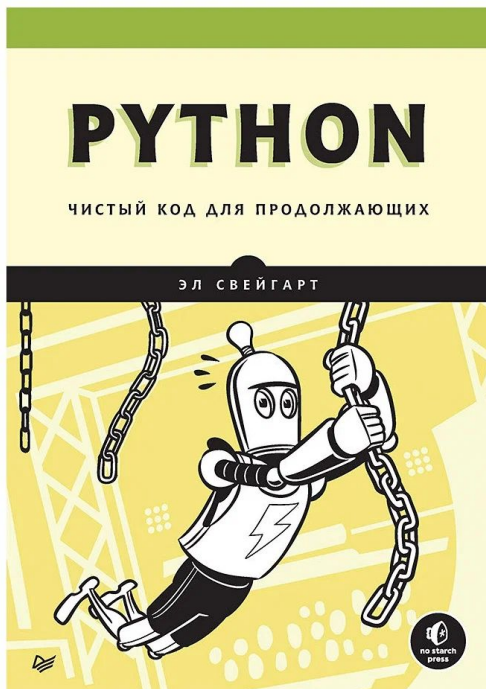
Дублирование кода – пустая трата времени и ресурсов. Тебе придется поддерживать одну и ту же логику и тестировать код сразу в двух местах, причем если ты изменишь код в одном месте, его нужно будет изменить и в другом.

В большинстве случаев дублирование кода происходит из-за незнания системы. Прежде чем что-либо писать, прояви прагматизм: осмотрись. Возможно, эта функция где-то реализована. Возможно, эта бизнес-логика существует в другом месте. Повторное использование кода – всегда разумное решение.

import this



Принципы написания кода



Паттерны проектирования



Паттерны проектирования

Порождающие

Отвечают за удобное и безопасное создание новых объектов или даже целых семейств объектов.

Структурные

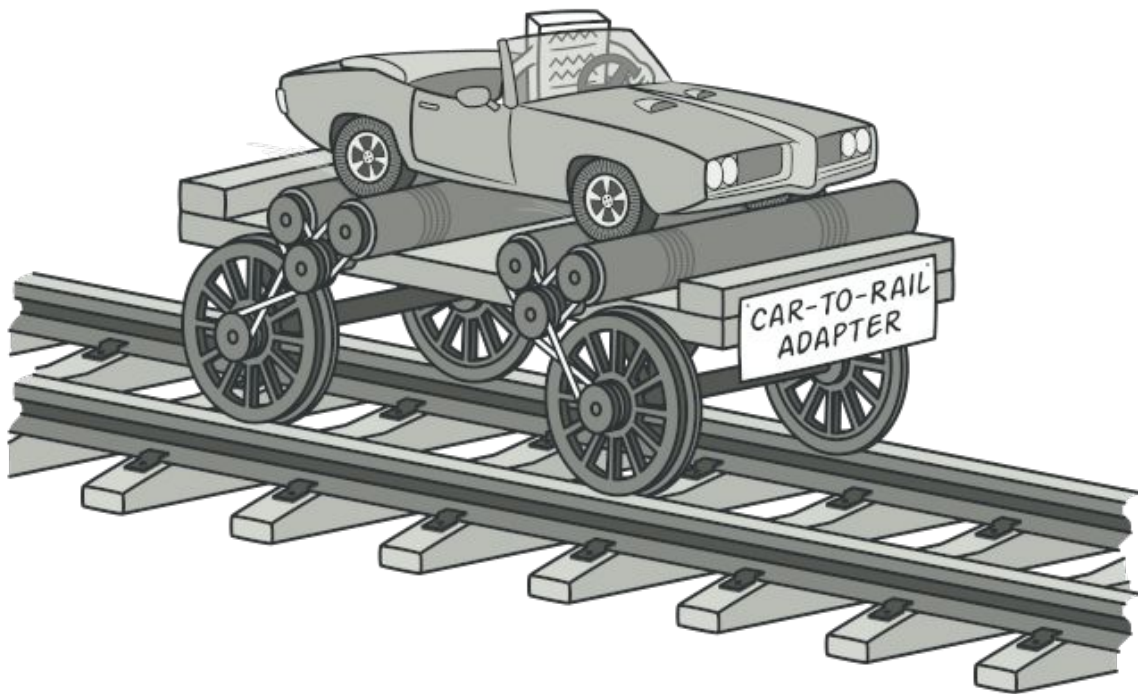
Отвечают за построение удобных в поддержке иерархий классов.

Поведенческие

Решают задачи эффективного и безопасного взаимодействия между объектами программы.

Адаптер

Структурный паттерн проектирования, который позволяет объектам с несовместимыми интерфейсами работать вместе.



Адаптер

Структурный паттерн проектирования, который позволяет объектам с несовместимыми интерфейсами работать вместе.

```
class Rails:
    def drive(self) -> str:
        return "choo-choo"

class Car:
    def move(self) -> str:
        return "Vroom vroom"

class Adapter:
    def __init__(self, rails, car):
        self.rails = rails
        self._car = car

    def drive(self) -> str:
        return f"We do it on rails: {self._car.move()}"
```

Стратегия

Поведенческий паттерн, выносит набор алгоритмов в собственные классы и делает их взаимозаменяемыми



Стратегия

Поведенческий паттерн, выносит набор алгоритмов в собственные классы и делает их взаимозаменяемыми

```
def compress_file():
    args = parse_args()
    filename = pathlib.Path(args.filename)
    compressor = NoCompressionCompressor()

    if args.compress_with == 'gzip':
        compressor = GzipCompressor(args.compression_level)
    elif args.compress_with == 'bz2':
        compressor = BZ2Compressor(args.compression_level)

    manager = FileManager(filename, compressor)
    data = manager.read()
    manager.write(
        filename.with_suffix(compressor.get_suffix()),
        data,
    )
```

Фабричный метод

Порождающий паттерн проектирования, который определяет общий интерфейс для создания объектов в суперклассе, позволяя подклассам изменять тип создаваемых объектов.



Фабричный метод

```
class Transport(ABC):  
    @abstractmethod  
    def move(self):  
        pass  
  
class Airplane(Transport):  
    def move(self):  
        print("I believe I can fly")  
  
class Car(Transport):  
    def move(self):  
        print("I'm on the highway to hell.")
```

Фабричный метод

```
class Transport(ABC):
    @abstractmethod
    def move(self):
        pass

class Airplane(Transport):
    def move(self):
        print("I believe I can fly")

class Car(Transport):
    def move(self):
        print("I'm on the highway to hell.")
```

```
class Creator(ABC):
    @abstractmethod
    def factory_method(self) -> Transport:
        pass

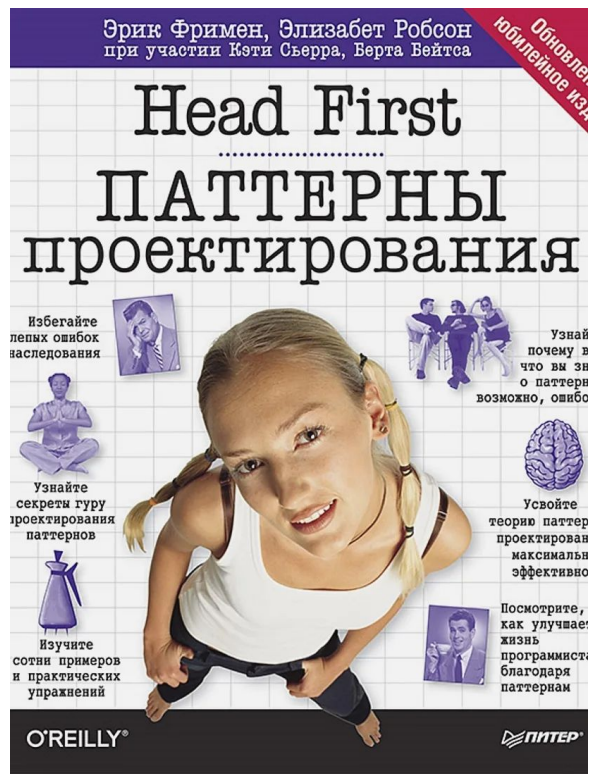
    def shipping(self):
        transport = self.factory_method()
        transport.move()

class AirplaneCreator(Creator):
    def factory method(self) -> Transport:
        return Airplane()

class CarCreator(Creator):
    def factory method(self) -> Transport:
        return Car()
```

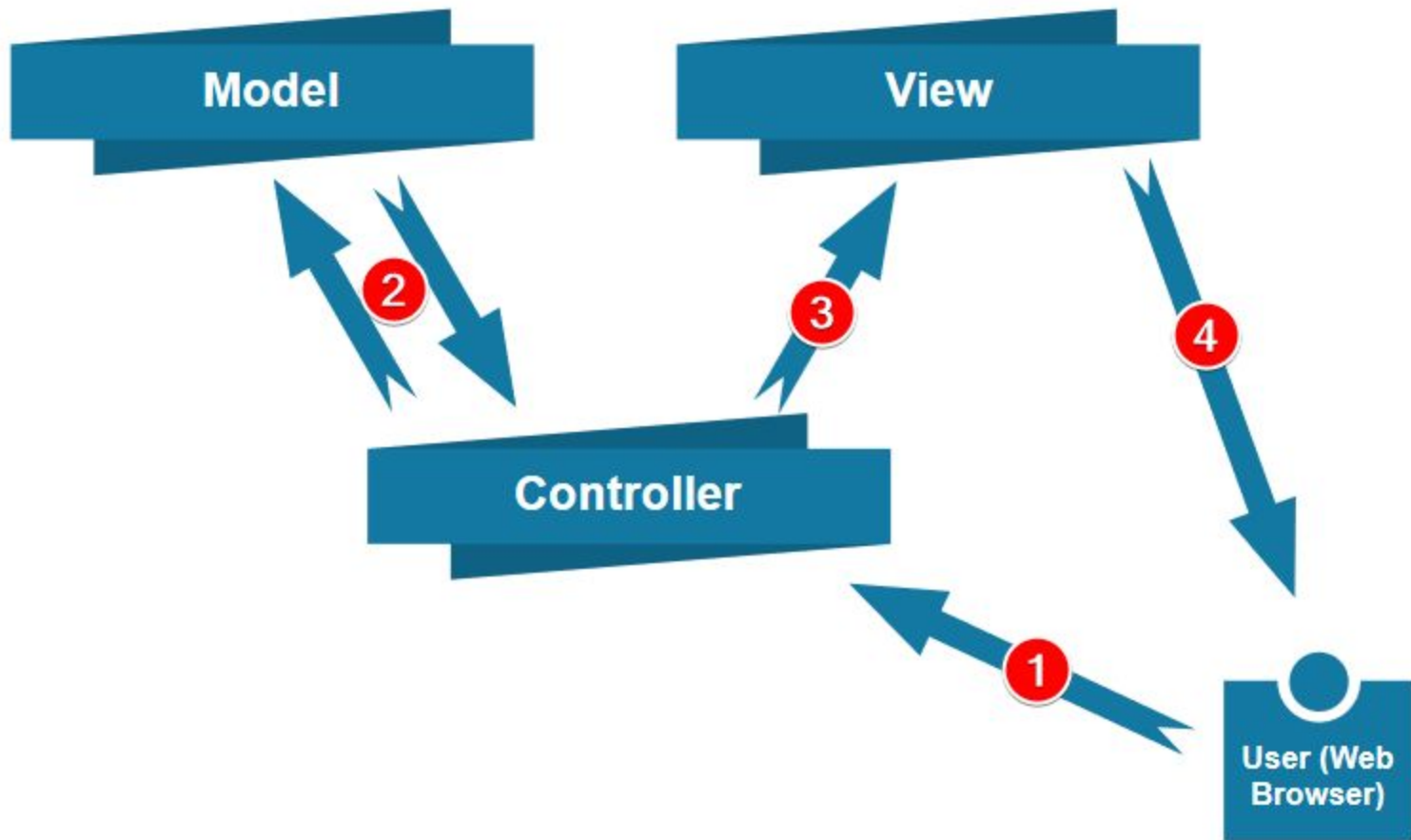
```
acdc car = CarCreator()
acdc car.shipping()
# I'm on the highway to hell.
```

Паттерны проектирования



Трехслойная архитектура





Трехслойная архитектура

Работа с данными

Все про работу с базой данных

[
получить список товаров из БД,
отсортированных по
возрастанию цены
]

Бизнес-логика

Функции, которые реализуют
логику приложения и
многократно используются в
представлениях

[
при создании товара, проверять
существует ли бренд товара в
БД. Если не существует, то
создать новый бренд
]

Представление

Простые методы, которые
отвечают за ответ клиента по
тому или иному протоколу

[
сформировать для клиента json-
объект:
- в поле data положить список,
состоящий из 3 полей модели
- в поле meta указать
количество отданных товаров
]

Архитектура кода на Python

YOUNG & YANDEX

Дамир Гарифуллин
Бэкенд-разработчик карт офисов

